

Functions, Procedures, Triggers, and Views in SQL

Base de Datos 2026

Funciones y Procedimientos

- Un **store procedure/function** es un conjunto de sentencias SQL con un nombre asignado y puede ser reutilizado cuantas veces se desee.
- Generalización de SQL añadiendo la estructura de un lenguaje de programación al lenguaje SQL

Estructura típicamente disponibles en procedimientos almacenados:

- Variables Locales
- Sentencias IF
- Sentencias LOOP

Funciones

```
create function dept_count(dept_name varchar(20))  
  returns integer  
  begin  
    declare d_count integer;  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name= dept_name  
  return d_count;  
  end
```

```
select dept_name, budget  
from instructor  
where dept_count(dept_name) > 12;
```

Procedimientos

```
create procedure dept_count_proc(in dept_name varchar(20),  
                                out d_count integer)  
begin  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name= dept_count_proc.dept_name  
end  
  
declare d_count integer;  
call dept_count_proc('Physics', d_count);
```

Procedimientos

Modos de Pasajes de Parámetros:

- **IN**: parámetro pasado por valor
- **OUT**: parámetro pasado por referencia
- **INOUT**: parámetro pasado por referencia y puede ser modificado pero se asume que ha sido inicializado

Sentencias **While** y **Repeat**

```
while boolean expression do  
    sequence of statements;  
end while
```

```
repeat  
    sequence of statements;  
until boolean expression  
end repeat
```

For loop...

```
declare n integer default 0;  
for r as  
    select budget from department  
    where dept_name = 'Music'  
do  
    set n = n - r.budget  
end for
```

Sentencia if-then-else

```
if boolean expression  
    then statement or compound statement  
elseif boolean expression  
    then statement or compound statement  
else statement or compound statement  
end if
```

Ejemplo: registrar un estudiante en un curso

```
create function registerStudent(  
    in s_id varchar(5),  
    in s_courseid varchar (8),  
    in s_secid varchar (8),  
    in s_semester varchar (6),  
    in s_year numeric (4,0),  
    out errorMsg varchar(100)  
returns integer  
begin  
    declare currEnrol int;  
    select count(*) into currEnrol  
    from takes  
    where course_id = s_courseid and sec_id = s_secid  
    and semester = s_semester and year = s_year;  
    declare limit int;  
    select capacity into limit  
    from classroom natural join section  
    where course_id = s_courseid and sec_id = s_secid  
    and semester = s_semester and year = s_year;
```

Ejemplo: registrar un estudiante en un curso...

```
if (currEnrol < limit)
    begin
        insert into takes values
            (s_id, s_courseid, s_secid, s_semester, s_year, null);
        return(0);
    end
-- Otherwise, section capacity limit already reached
set errorMsg = 'Enrollment limit reached for course ' || s_courseid
    || ' section ' || s_secid;
return(-1);
end;
```

Funciones o procedimientos?

Propósito principal:

- **Procedimientos Almacenados (Stored Procedures):** Se utilizan principalmente para **realizar operaciones en una base de datos**, como insertar, actualizar o eliminar registros, así como para realizar tareas lógicas o de flujo de control dentro de la base de datos. Los procedimientos almacenados pueden realizar cambios en los datos de la base de datos y también pueden devolver valores mediante parámetros de salida.
- **Funciones (Functions):** se utilizan principalmente para **realizar cálculos o transformaciones en los datos y devolver un valor** basado en los parámetros de entrada. No pueden realizar modificaciones en la base de datos, como insertar o actualizar registros.

Funciones o procedimientos?

Retorno de valor:

- **Procedimientos Almacenados:** Los procedimientos almacenados pueden tener parámetros de salida para devolver valores, pero **no necesariamente devuelven un valor**. Pueden realizar cambios en la base de datos y no están diseñados específicamente para devolver un resultado único.
- **Funciones:** Las funciones están **diseñadas específicamente para devolver un valor**, ya sea un valor escalar (como un número o una cadena) o una tabla (en el caso de funciones de tabla). No pueden realizar modificaciones en la base de datos.

Funciones o procedimientos?

Uso en consultas:

- **Procedimientos Almacenados:** Los procedimientos almacenados se llaman utilizando la instrucción EXEC o CALL y se utilizan en las consultas como si fueran comandos SQL independientes.
- **Funciones:** Las funciones se utilizan en las consultas como expresiones, y su valor devuelto se puede usar directamente en una consulta SELECT u otras partes de una consulta SQL.

Críticas a funciones y procedimientos

- **Portabilidad:** Los procedimientos almacenados pueden no ser portables entre sistemas de gestión de bases de datos, lo que dificulta la migración.
- **Dificultad de mantenimiento:** A medida que crecen en complejidad, los procedimientos almacenados pueden volverse difíciles de mantener y depurar.
- **Escalabilidad:** Un uso excesivo de procedimientos almacenados puede afectar negativamente la escalabilidad de una aplicación.
- **Seguridad:** Los procedimientos almacenados mal diseñados pueden presentar riesgos de seguridad en la base de datos.
- **Complejidad de desarrollo:** Desarrollar procedimientos almacenados puede ser más complejo que escribir código en lenguajes de programación de aplicaciones.
- **Acoplamiento de la base de datos:** El uso excesivo de procedimientos almacenados puede acoplar estrechamente la lógica de negocio a la base de datos, lo que dificulta los cambios y la flexibilidad.

Triggers...

- Un disparador o **trigger** es una funcionalidad asociada a una tabla que el motor ejecuta de forma **automática** en reacción a un **INSERT**, **UPDATE**, o **DELETE** sobre esa tabla.
- Se puede configurar un trigger para que se active o bien antes (**BEFORE**) o después (**AFTER**) del evento del trigger

Triggers (Sintaxis)

CREATE TRIGGER trigger_name **trigger_time** trigger_event

ON table_name **FOR EACH ROW**

BEGIN

[trigger_order]

trigger_body

END;

trigger_time: {BEFORE | AFTER}

trigger_event: {INSERT | UPDATE | DELETE}

trigger_order: {FOLLOWS| PRECEDES} other_trigger_name

Ejemplo

```
create trigger credits_earned after update of takes on (grade)  
referencing new row as nrow  
referencing old row as orow  
for each row  
when nrow.grade <> 'F' and nrow.grade is not null  
  and (orow.grade = 'F' or orow.grade is null)  
begin atomic  
  update student  
  set tot_cred = tot_cred +  
    (select credits  
     from course  
     where course.course_id = nrow.course_id)  
  where student.id = nrow.id;  
end;
```

Views

- Una vista (view) es una consulta a la que se le pone un nombre para poder reutilizarla.
- Genera una **tabla virtual**: No almacena ningún dato en sí, sino que se ejecuta la consulta cada vez que se usa la vista.
- Las vistas son útiles para ocultar información innecesaria y para recopilar información de más de una relación en una sola vista.
- Cuando se define una vista, normalmente la base de datos almacena sólo la consulta que define la vista.

Views

Definición:

CREATE VIEW v AS <query expression>;

Ejemplo

```
create view faculty as  
select ID, name, dept_name  
from instructor;
```

Otro Ejemplo

```
create view physics_fall_2009 as  
  select course.course_id, sec.id, building, room.number  
  from course, section  
  where course.course_id = section.course_id  
        and course.dept_name = 'Physics'  
        and section.semester = 'Fall'  
        and section.year = '2009';
```

```
select course_id  
from physics_fall_2009  
where building = 'Watson';
```

Materialized Views

- Una vista materializada almacena el resultado de una consulta específica como una tabla física en la base de datos.
- Los datos están precalculados y guardados, no se vuelve a ejecutar la consulta cada vez que se accede a la vista materializada.
- Hay que tener cuidado de mantener la vista actualizada.

Materialized Views

Definición:

CREATE MATERIALIZED VIEW v AS <query expression>;

Materialized Views

- Las vistas materializadas constituyen datos redundantes
- Sin embargo, en muchos casos es mucho más barato leer el contenido de una vista materializada que calcular el contenido de la vista ejecutando la consulta que define la vista.

Materialized Views

Las vistas materializadas son importantes para mejorar el rendimiento en algunas aplicaciones.

```
create view department_total_salary(dept_name, total_salary) as  
select dept_name, sum (salary)  
from instructor  
group by dept_name;
```

Materialized Views

- Suponga que el monto de salario total en un departamento se requiere con frecuencia.
- Calcular la vista requiere leer cada tupla de instructor perteneciente a un departamento y resumir los montos de los salarios, lo que puede llevar mucho tiempo.
- Por el contrario, si la definición de la vista del monto del salario total se **materializara**, el monto del salario total podría encontrarse buscando una tupla en la vista materializada